



Data Build Tool — Complete Interview Guide

Fundamentals · Materializations · Jinja · Testing · Deployment

80 Interview Questions with Answers · Snowflake Integration · Cheat Sheet

CHAPTER 1

dbt Fundamentals

What dbt does · Project structure · ref() · source() · CLI commands

1.1 What dbt Does and Why It Matters

dbt (data build tool) is an open-source transformation framework that lets data engineers and analytics engineers write modular SQL SELECT statements managed as version-controlled code. dbt handles the T in ELT — it transforms data that is already in your warehouse. It never loads data from external sources.

Dimension	Traditional ETL	dbt (ELT)
Where transforms run	ETL server (Informatica, Talend, SSIS)	Inside the warehouse (Snowflake, BigQuery, Redshift)
Language	Proprietary drag-and-drop or Python	SQL with Jinja templating
Version control	Often absent or partial	Full Git workflow — branches, pull requests, code review
Testing	Separate QA process, often manual	Built-in: unique, not_null, accepted_values, relationships
Documentation	Often outdated	Auto-generated from code via dbt docs generate
Dependencies	Manual or complex configuration	Automatic via ref() function
Dev isolation	Hard to achieve	Profiles and targets make it straightforward

1.2 dbt Project Structure

Directory Layout

```

my_dbt_project/
  dbt_project.yml    - project name, version, model paths, default configs
  packages.yml       - external packages (dbt-utils, dbt-expectations)
  profiles.yml       - warehouse credentials (NOT committed to git)

  models/
    staging/         - one model per source table; type-cast and rename only
      _sources.yml   - declares external (raw) tables via source()
      _staging.yml   - tests and documentation for staging models
      stg_orders.sql
    intermediate/    - complex joins and business logic
    marts/
      dimensions/    - dim_customers, dim_products, dim_stores
      facts/         - fct_orders, fct_order_items
      gold/          - pre-aggregated KPIs, Customer 360

  snapshots/        - SCD2 history tracking
  tests/            - singular test SQL files
  macros/           - reusable Jinja functions
  seeds/            - small CSV files loaded as tables
  target/           - compiled output (NOT in git)

```

1.3 ref() and source() — The Core Functions

Function	Used For	What It Does	Never Use Instead
<code>ref('model_name')</code>	Any model dbt created	Resolves to the correct schema for the active target environment. Registers the upstream model as a DAG dependency.	Hardcoded schema names like <code>dev_db.public.stg_orders</code>
<code>source('name','table')</code>	Raw/external tables dbt did not create	Points to declared source in <code>_sources.yml</code> . Enables source freshness checks and lineage tracking.	Hardcoded raw table names

KEY INSIGHT

`ref()` serves two purposes: it resolves the correct schema per target environment (dev vs prod) AND it registers the de

1.4 CLI Commands Reference

Command	What It Does	When to Use
<code>dbt run</code>	Compiles and executes models in DAG order	Main transformation command
<code>dbt test</code>	Runs all configured tests	After <code>dbt run</code> to validate data quality
<code>dbt build</code>	<code>dbt run</code> plus <code>dbt test</code> together in DAG order	Production — run and test in one command
<code>dbt compile</code>	Renders Jinja, writes to <code>target/compiled</code> , does NOT execute	Debug Jinja or <code>ref()</code> issues
<code>dbt source freshness</code>	Checks source freshness against <code>loaded_at_field</code>	Verify upstream data is current
<code>dbt snapshot</code>	Executes snapshot blocks for SCD2	Separate from <code>dbt run</code> — must be scheduled separately
<code>dbt docs generate</code>	Builds <code>manifest.json</code> and <code>catalog.json</code> for docs site	Before <code>dbt docs serve</code>
<code>dbt deps</code>	Installs packages from <code>packages.yml</code>	After cloning project or adding packages
<code>dbt ls</code>	Lists models matching a selector without running	Understand what a selector covers
<code>dbt debug</code>	Tests warehouse connection	Diagnose connection or profile issues

CHAPTER 2

Materializations

view · table · incremental · ephemeral · snapshot

2.1 The Four Materializations

Materialization	Creates	On Every Run	Best For	Avoid When
view	Database view — no data stored	Drops and recreates view definition only	Staging models, lightweight transforms, rarely-queried models	Frequently queried models — query re-executes SQL every time
table	Physical table	Drops and recreates entire table from scratch	Final marts, aggregations, models queried by BI tools	Data grows very large — full rebuild becomes slow
incremental	Physical table that grows	Appends or merges only rows newer than last run	Large fact tables, event data, any table with a time dimension	Small tables — overhead exceeds benefit
ephemeral	Nothing — inlined as a CTE	Inlined into the referencing model's compiled SQL	Intermediate calculations that do not need to be queried directly	Models needed for debugging — you cannot SELECT from an ephemeral model

2.2 Incremental Models — Deep Dive

Incremental models are the most important materialization for production data engineering. Instead of rebuilding a 1TB fact table daily, incremental models process only new records. This is the most tested topic in dbt interviews.

Concept	Explanation
<code>is_incremental()</code>	Returns TRUE on runs 2 and beyond. Returns FALSE on first run and after <code>--full-refresh</code> . The WHERE clause inside this block is only applied when TRUE.
<code>unique_key</code>	The column(s) used for MERGE deduplication. Without <code>unique_key</code> , dbt appends rows. With it, dbt merges — updating matches and inserting new rows.
<code>{{ this }}</code>	Reference to the existing incremental table. Used in the filter: <code>WHERE date > (SELECT MAX(date) FROM {{ this }})</code> .
<code>on_schema_change</code>	Controls what happens when source adds or removes columns. Default is <code>ignore</code> (new columns silently dropped). Use <code>append_new_columns</code> in production.
<code>--full-refresh</code>	Forces drop and recreate. Use after schema changes, filter bugs, or data corrections. Can be prevented with <code>full_refresh: false</code> in <code>dbt_project.yml</code> for critical tables.

IMPORTANT

Late-arriving data: if your incremental filter is only `WHERE date > MAX(date)`, records that arrive late (yesterday's d

CHAPTER 3

Jinja Templating and Macros

Variables · Control flow · Macros · dbt-utils

3.1 Jinja Syntax in dbt

Syntax	Purpose	Example
<code>{{ expression }}</code>	Output a value	<code>{{ ref('stg_orders') }}</code> , <code>{{ var('start_date') }}</code> , <code>{{ this }}</code>
<code>{% statement %}</code>	Control flow — if/else, for loops, set variables	<code>{% if is_incremental() %}...{% endif %}</code>
<code>{# comment #}</code>	Jinja comment — not in compiled SQL	<code>{# TODO: add partition pruning #}</code>
<code>{{ var('name') }}</code>	Read project variable from <code>dbt_project.yml</code> or <code>--vars</code> CLI flag	<code>{{ var('start_date', '2020-01-01') }}</code> with default
<code>{{ target.name }}</code>	Current target name — dev or prod	<code>{% if target.name == 'prod' %}...{% endif %}</code>
<code>{{ target.schema }}</code>	Current target schema	Useful for building dynamic table paths

3.2 Macros — Reusable SQL Functions

Macros are Jinja functions defined in the `macros/` folder. They are called at compile time and injected into the model's SQL. Use them to eliminate copy-paste SQL across models.

Macro Pattern — cents_to_dollars

```
-- macros/cents_to_dollars.sql
{% macro cents_to_dollars(column_name, scale=2) %}
  ({{ column_name }} / 100)::NUMERIC(16, {{ scale }})
{% endmacro %}

-- Usage in any model:
SELECT
  order_id,
  {{ cents_to_dollars('amount_cents') }} AS amount_dollars,
  {{ cents_to_dollars('tax_cents', scale=4) }} AS tax_dollars
FROM {{ ref('stg_orders') }}
```

3.3 dbt-utils Essential Macros

Macro	What It Does	Example Use
<code>generate_surrogate_key(list)</code>	MD5 hash of multiple columns for surrogate key	<code>{{ dbt_utils.generate_surrogate_key(['order_id', 'line_item_id']) }}</code>
<code>date_spine(datepart, start, end)</code>	Generate a row per date between two dates	Fill gaps in daily aggregations
<code>star(ref, except)</code>	SELECT all columns except the ones listed	<code>{{ dbt_utils.star(ref('stg_orders'), except=['pii_col']) }}</code>
<code>pivot(col, values)</code>	Turn row values into columns dynamically	Turn <code>payment_method</code> rows into one column per method

expression_is_true(expr)	Generic test — fails if expression is false	expression_is_true: expression: ">= 0" on amount column
--------------------------	---	---

CHAPTER 4

Testing Strategy

Generic tests · Singular tests · Severity · Source freshness

4.1 Built-in Generic Tests

Generic tests are configured in schema.yml files and apply to columns declaratively. dbt generates the test SQL automatically. The test passes if it returns 0 rows.

Test	What It Checks	Failure Meaning	Fix Pattern
unique	No two rows share the same value	Duplicate records in the column	Find duplicates: <code>SELECT col, COUNT(*) GROUP BY 1 HAVING COUNT(*) > 1</code>
not_null	No NULL values in the column	NULLs introduced in source or transform	<code>SELECT * FROM model WHERE col IS NULL LIMIT 10</code>
accepted_values	All values are in the specified list	New value appeared in source	<code>SELECT DISTINCT col FROM model WHERE col NOT IN ('val1','val2')</code>
relationships	Every value exists in the referenced column	Orphaned foreign key	Find orphans: <code>LEFT JOIN reference table WHERE ref_col IS NULL</code>

4.2 Singular Tests

Singular tests are SQL files in the tests/ folder. You write the full query. The test passes if the query returns 0 rows. The test fails if it returns any rows. Write the query to return the problematic records.

Singular Test Examples

```
-- tests/assert_ship_after_order.sql
-- Returns rows where ship_date is before order_date (impossible)
SELECT order_id, order_date, ship_date
FROM {{ ref('fct_orders') }}
WHERE ship_date < order_date
  AND ship_date IS NOT NULL

-- tests/assert_revenue_positive.sql
-- Returns months where total revenue is zero or negative
SELECT DATE_TRUNC('month', order_date) AS month, SUM(total_amount) AS rev
FROM {{ ref('fct_orders') }}
GROUP BY 1
HAVING SUM(total_amount) <= 0
```

4.3 Test Severity Configuration

Setting	Pipeline Behavior	Use For
severity: error (default)	Pipeline stops. Exit code 1. CI fails.	PK uniqueness, not_null on critical keys, FK integrity
severity: warn	Test result logged. Pipeline continues. Exit code 0.	Soft data quality checks where some violations are acceptable

warn_if: >= N	Warn only when N or more rows fail	Allow a small number of exceptions before alerting
error_if: >= N	Error only when N or more rows fail	Graduated response — warn at 1 failure, error at 100 failures

CHAPTER 5

Snapshots and Deployment

SCD2 · Profiles · Slim CI · --defer flag

5.1 dbt Snapshots — SCD Type 2

dbt snapshots automatically implement SCD2. When a tracked column changes, dbt expires the old row (sets `dbt_valid_to`) and inserts a new current row. Four columns are added by dbt automatically.

Column Added	Contents	Query Pattern
<code>dbt_scd_id</code>	MD5 hash — unique identifier for each snapshot row	Use as surrogate key when joining from fact tables
<code>dbt_valid_from</code>	Timestamp when this version became active	Lower bound in PIT join: <code>event_date >= dbt_valid_from</code>
<code>dbt_valid_to</code>	Timestamp when superseded. NULL = current record.	Filter: <code>WHERE dbt_valid_to IS NULL</code> to get current state
<code>dbt_updated_at</code>	When dbt last processed this row	Used internally for dedup and ordering

KEY INSIGHT

The most important snapshot query pattern is the point-in-time join from a fact table:

`JOIN snapshot_table s ON s.natural_key = f.natural_key`

`AND f.event_date BETWEEN s.dbt_valid_from AND COALESCE(s.dbt_valid_to, '9999-12-31')`

5.2 Profiles and Environment Isolation

`profiles.yml` defines multiple targets (dev, staging, prod) with different warehouse credentials. `ref()` automatically resolves to the correct schema for the active target. `profiles.yml` is never committed to git — credentials stay local or in a secrets manager.

Target Config	Dev	Prod
database	DEV_DB	PROD_DB
schema	dbt_alice (per developer)	public (shared)
warehouse	TRANSFORM_WH_DEV	TRANSFORM_WH_PROD
threads	4	16
auth method	username + password	RSA private key (key pair)
query_tag	dbt_dev	dbt_prod

5.3 Slim CI — Only Test What Changed

Slim CI compares the current branch's `manifest.json` against the production manifest to identify only models that changed. It runs those models plus their downstream dependents, making CI 10 to 100 times faster than running all models.

Flag	Purpose
------	---------

--select state:modified+	Run only changed models plus everything downstream of them
--defer	Use prod versions of unmodified upstream models instead of building them in CI
--state ./prod_manifest	Tell dbt where the previous production manifest is for comparison
--target ci	Use the CI target defined in profiles.yml (separate schema per PR)

TIP

The --defer flag is also useful for local development. With --defer --state ./prod_target, a developer can test only t

Interview Questions — 30 with Answers

Q1. What is dbt and what problem does it solve?

EASY

Answer:

dbt is an open-source SQL transformation framework that brings software engineering practices to data transformation.

The problem it solves: traditional ETL requires separate compute infrastructure and proprietary tools. dbt uses the warehouse's own compute for transformations, bringing version control, testing, documentation, and CI/CD to SQL-based data work.

KEY INSIGHT

dbt does the T in ELT. It never loads data — it transforms data already in your warehouse.

Q2. What is the difference between `ref()` and `source()`?

MEDIUM

Answer:

`ref()` references models that dbt created. It resolves to the correct schema per target environment and registers the upstream dependency in the DAG.

`source()` references raw or external tables that dbt reads from but did not create. Use `source()` in staging models to reference raw landing tables.

`source()` enables source freshness checks and adds the raw table to the dbt lineage graph.

KEY INSIGHT

Simple rule: if dbt created it, use `ref()`. If it came from outside dbt, use `source()`.

Q3. Explain the four materializations and when you would use each.

MEDIUM

Answer:

view: a database view — no data stored. Use for staging models and lightweight transforms.

table: a physical table rebuilt on every run. Use for mart models queried by BI tools.

incremental: a physical table that grows. Use for large fact tables and event data.

ephemeral: inlined as a CTE, never persisted. Use for intermediate calculations.

KEY INSIGHT

staging=view, marts=table, large facts=incremental, intermediate logic=ephemeral.

Q4. How do incremental models work on first run versus subsequent runs?

HARD

Answer:

First run: `is_incremental()` returns FALSE. The WHERE clause inside the block is skipped entirely. All rows are processed and loaded into a new table.

Subsequent runs: `is_incremental()` returns TRUE. The WHERE clause activates and filters to only rows newer than the max already in the table.

With `unique_key` configured, dbt generates a MERGE — updating matched rows and inserting new ones. Without `unique_key`, dbt appends rows.

KEY INSIGHT

`is_incremental()` is True on runs 2+. False on first run and `--full-refresh`.

Q5. What is `on_schema_change` and which value should you use in production?**MEDIUM**

Answer:

`on_schema_change` controls what happens when your incremental model's source adds or removes columns.

`ignore` (default): new source columns are silently dropped from incremental runs. Dangerous — silent data loss.

`append_new_columns`: adds new source columns to the target table. Best for most production cases.

`sync_all_columns`: target exactly mirrors source — removes columns that disappeared from source.

`fail`: any schema change raises an error — forces explicit handling.

KEY INSIGHT

Never use the default in production. Set `append_new_columns` to prevent silent data loss.

Q6. What is a dbt snapshot and how does it implement SCD2?**HARD**

Answer:

A dbt snapshot is a block defined in the `snapshots/` folder with a config that specifies `unique_key`, `strategy` (timestamp or check), and `target_schema`.

When the strategy detects a change — either the `updated_at` column changed (timestamp) or a checked column's hash changed (check) — dbt adds `dbt_valid_to` to the old row and inserts a new row with `dbt_valid_from = now` and `dbt_valid_to = NULL`.

The most important query pattern: join facts to the snapshot using `event_date BETWEEN dbt_valid_from AND COALESCE(dbt_valid_to, '9999-12-31')` to get the dimension version active when the event occurred.

KEY INSIGHT

Snapshot strategy: timestamp uses `updated_at` column. Check strategy hashes specified columns. Always use timestamp when possible.

Q7. What is Slim CI and what three flags does it require?**HARD**

Answer:

Slim CI compares the current branch's `manifest.json` against the production manifest to identify only changed models. It runs those models plus their downstream dependents.

The three required flags: `--select state:modified+` (run changed plus downstream), `--defer` (use prod data for unmodified upstreams), `--state path/to/prod/target` (where the prod manifest lives).

This makes CI 10-100x faster because a PR changing one staging model only re-runs that model and its downstream, not all 200 models.

KEY INSIGHT

`state:modified+ --defer --state` = the three Slim CI flags. All three required together.

Q8. What is a singular test and how does it differ from a generic test?**MEDIUM****Answer:**

Singular test: a SQL file in tests/. You write the full query. The test PASSES if the query returns 0 rows. FAILS if it returns any rows.

Generic test: configured in schema.yml on a column. You specify parameters, dbt generates the SQL.

Generic tests are reusable across columns. Singular tests express complex business rules that column-level generics cannot.

KEY INSIGHT

Write singular tests to return the FAILING records. Zero rows = pass. Any rows = fail.

Q9. How do you configure test severity and when do you use warn vs error?**MEDIUM****Answer:**

severity: error (default) — test failure stops the pipeline. Exit code 1. CI fails. Use for primary key uniqueness, not_null on critical foreign keys, FK integrity.

severity: warn — test result is logged but pipeline continues. Exit code 0. Use for soft data quality checks where some violations are expected or acceptable.

You can also use warn_if and error_if with row count thresholds for graduated responses.

KEY INSIGHT

Error = stop pipeline. Warn = log and continue. Default is error. Always be explicit.

Q10. How does the --defer flag work and why is it useful for development?**HARD****Answer:**

--defer tells dbt to use production versions of models that are NOT in your current selection, instead of building them in your dev schema.

Example: you changed dim_customers. With --defer and --state pointing at prod, dbt runs dim_customers in your dev schema but uses the production version of stg_customers as its input — without you needing to build stg_customers first.

This means developers only need to build the models they changed, not the entire upstream graph. Saves significant dev time and warehouse cost.

KEY INSIGHT

--defer + --state = test your changes against production data without rebuilding everything upstream.

Quick Reference Cheat Sheet

CLI Commands

Command	Use Case
dbt run --select model+	Run model and all downstream models
dbt build --select state:modified+ --defer --state ./prod	Slim CI build
dbt test --select stg_orders	Run tests for one specific model
dbt compile --select my_model	Render Jinja without running SQL
dbt source freshness	Check all configured source freshness
dbt snapshot	Run all snapshot blocks for SCD2
dbt run --select my_model --full-refresh	Force drop and recreate incremental model
dbt docs generate && dbt docs serve	Build and browse the lineage documentation site
dbt deps	Install or update packages from packages.yml
dbt ls --select +model+	List the full neighborhood of a model

Materialization Decision Matrix

Scenario	Materialization	Key Config
Staging model for a source table	view	Default — no special config needed
Dashboard-facing aggregate, under 1GB	table	No special config needed
Fact table over 100GB growing daily	incremental	unique_key, on_schema_change, cluster_by
Intermediate CTE used in two models	ephemeral	Default — no special config needed
SCD2 customer dimension history	snapshot	strategy, unique_key, updated_at or check_cols
Small reference CSV like country codes	seed	column_types, schema in dbt_project.yml

Critical Facts — Know These Cold

Topic	The One Thing to Memorize
ref() purpose	Schema resolution per target environment AND DAG dependency registration
Incremental first run	is_incremental() is False — all rows loaded. Same as --full-refresh.
Late-arriving data	Add: OR date >= CURRENT_DATE minus N days as a lookback window
on_schema_change default	ignore — new columns silently dropped. Always set append_new_columns.
Singular test pass	Returns 0 rows = PASS. Write the query to return FAILING records.

Snapshot current query	WHERE dbt_valid_to IS NULL
Snapshot PIT join	event_date BETWEEN dbt_valid_from AND COALESCE(dbt_valid_to, '9999-12-31')
Slim CI three flags	state:modified+ and --defer and --state ./prod_manifest
copy_grants=True	Preserves Snowflake grants when a table is rebuilt
dbt build vs dbt run	dbt build = run plus test in DAG order. Use build in production.